

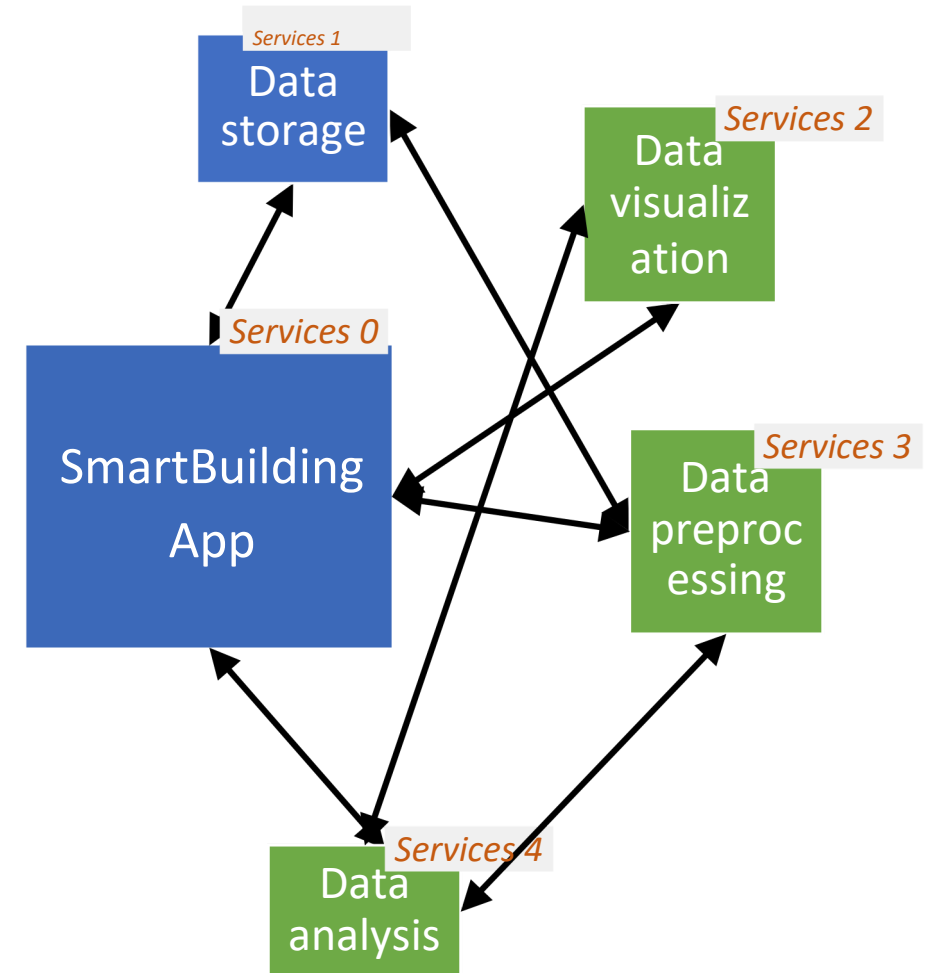
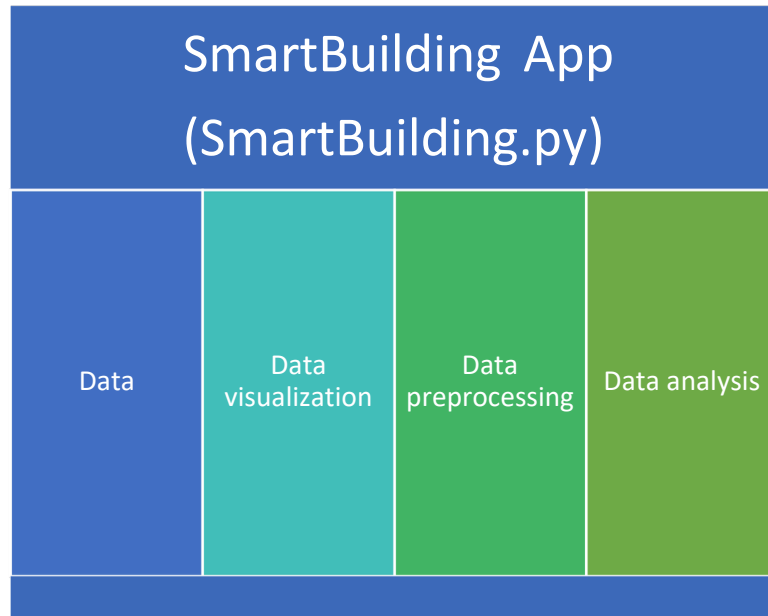
Microservice-based Application Development

Nazih Errahel

erraheln@ex.istu.edu

Monolithic -> Microservice Architecture

Example:



So how the microservices communicate ?

- Communication Types:

- **Synchronous :**

- HTTP/HTTPS is a synchronous protocol.
 - The client sends a request and waits for a response from the service.
 - client code can only continue its task when it receives the HTTP/HTTPS server response

- **Asynchronous :**

- Sender usually doesn't wait for a response.
 - Protocols like AMQP (a protocol supported by many operating systems and cloud environments) uses asynchronous messages.
 - E.g. RabbitMQ, Kafka

- One-to-one Communication

- One-to-many Communication

So how the microservices communicate ?

- Communication Types:

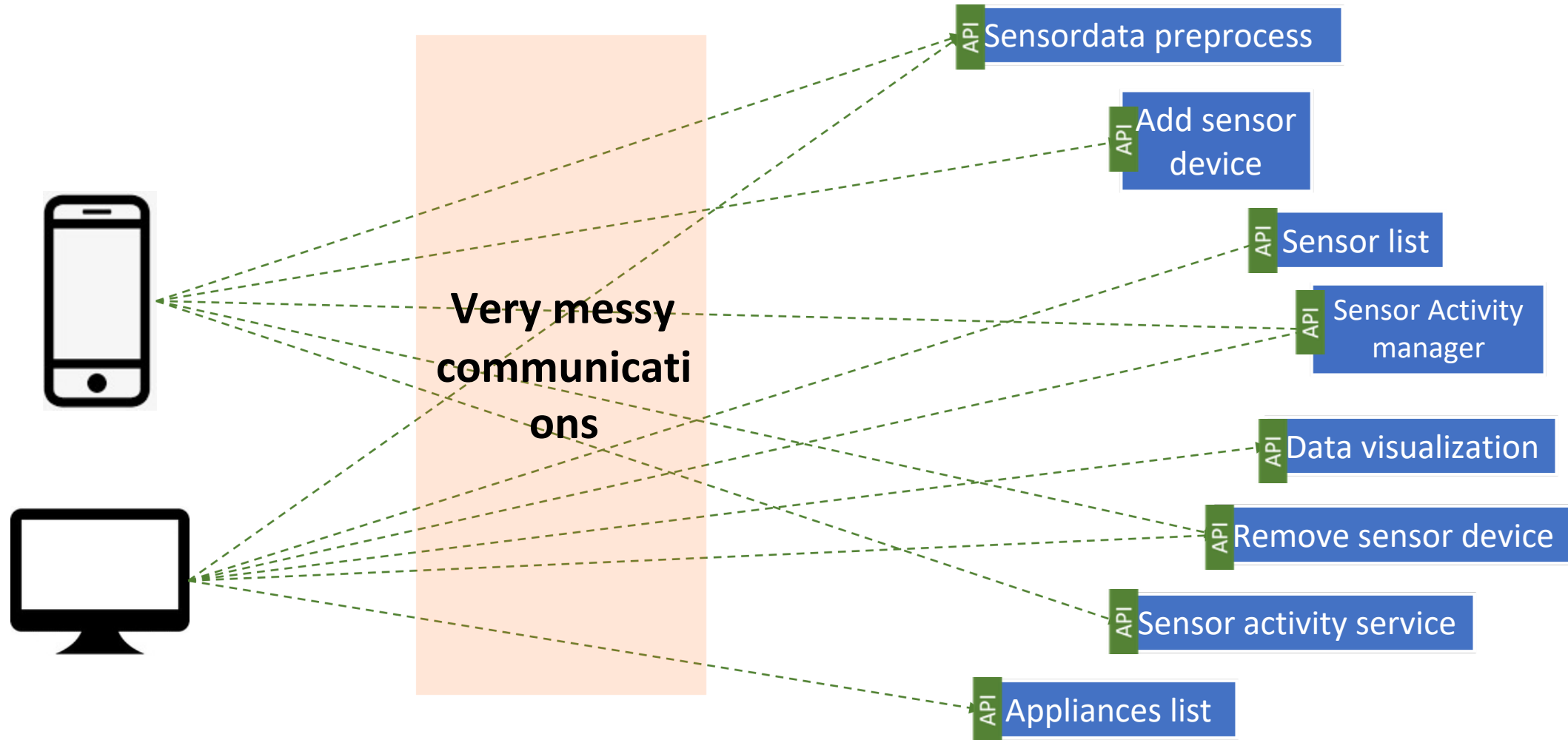
- One-to-one Communication

- A request must be processed by exactly one service (single producer and single receiver).
- E.g. Finding the min. temp of a particular date from the temp. sensor data
- Can be *synchronous communication*

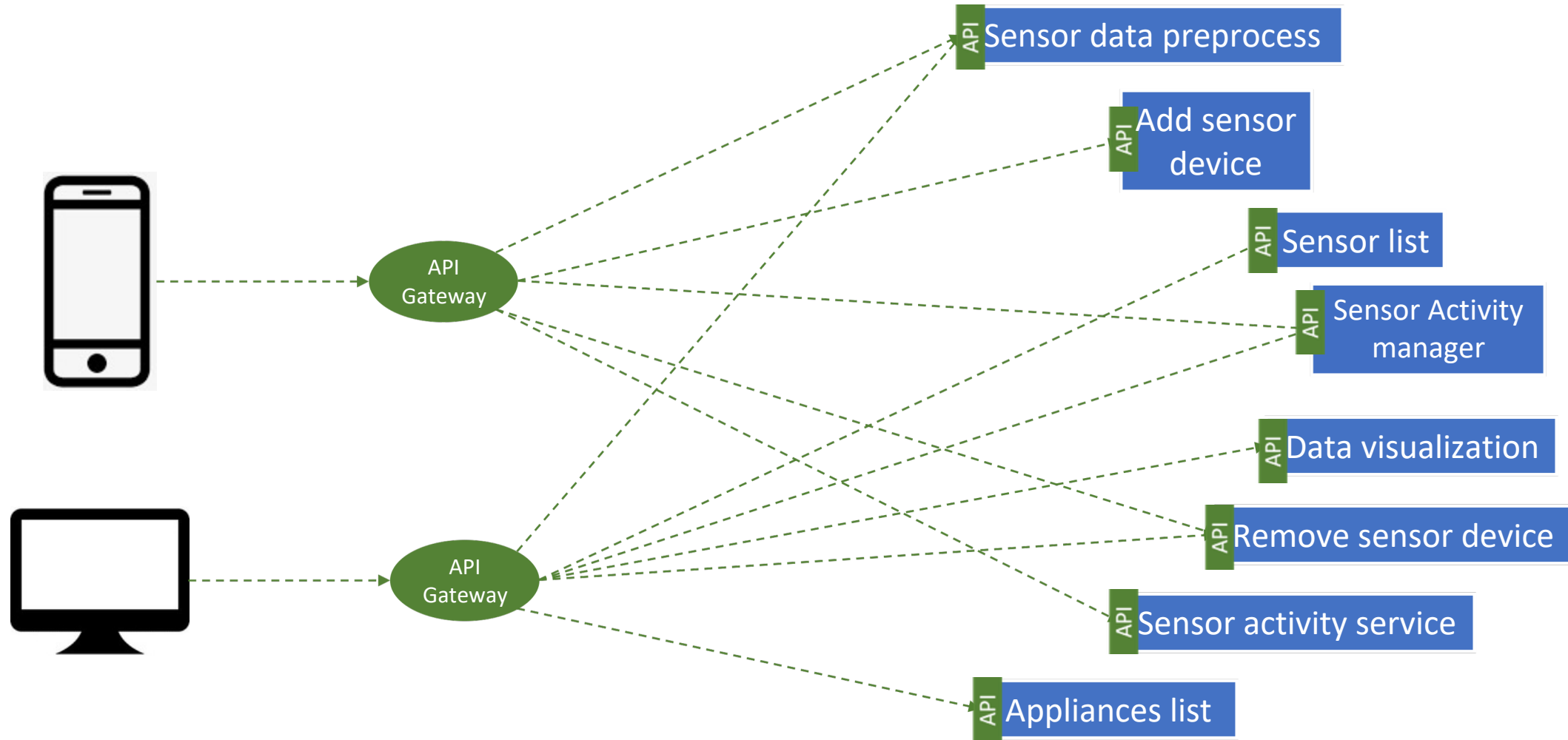
- One-to-many Communication

- Each request can be processed by zero to multiple receivers.
- Must be *asynchronous communication*
- Used in Event-driven architecture
- example is the publish/subscribe mechanism

Direct Client-to-MicroserviceCommunication : Without API Gateway



Client to microservicewith API Gateway



API Gateway

- An API Gateway is a server
- Single entry point into the system
- API Gateway encapsulates the internal system architecture
- provides an API that is tailored to each client
- Responsibilities:
 - Authentication
 - Monitoring
 - load balancing
 - request routing
 - Composition
 - protocol translation

API Gateway

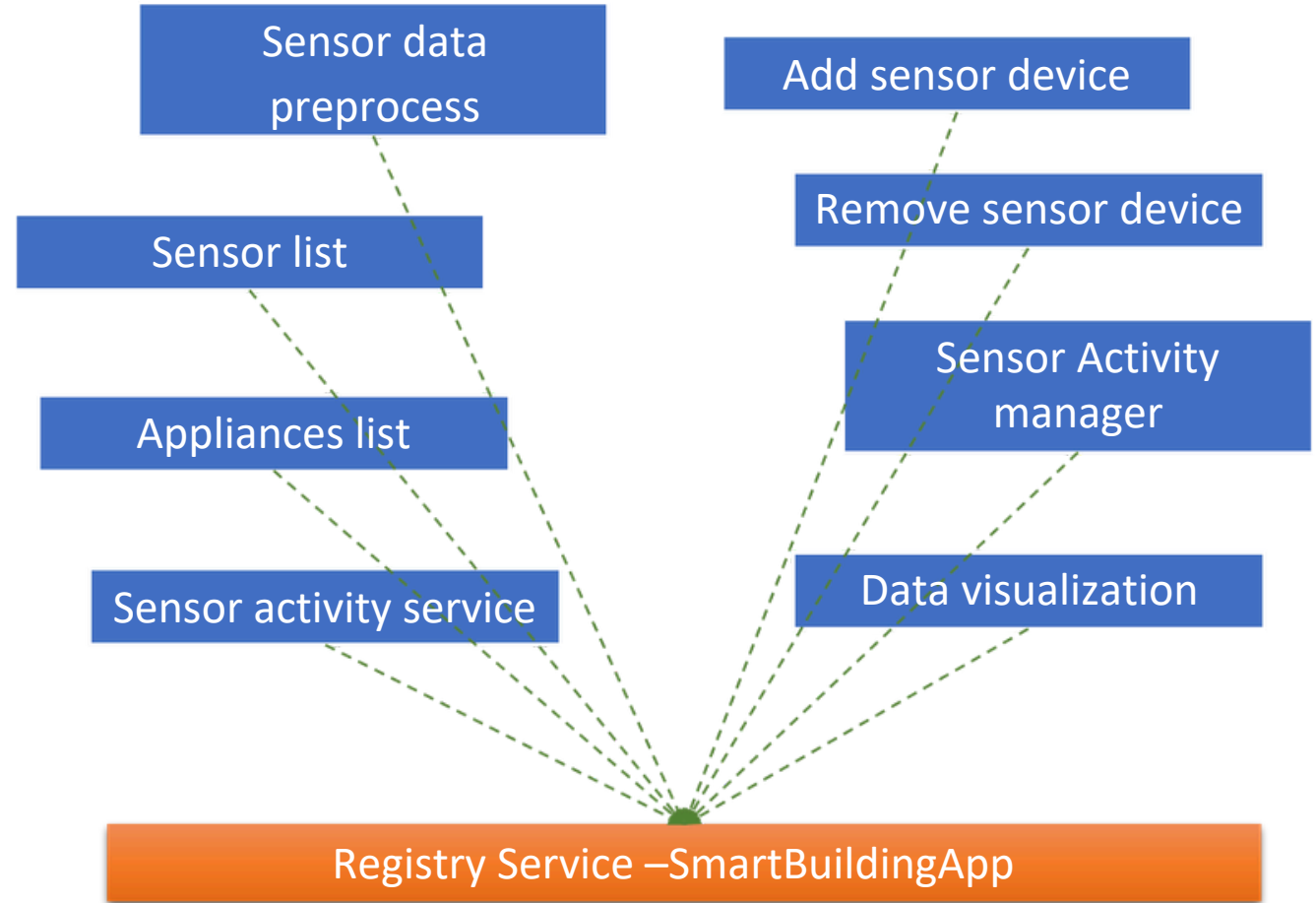
- May invoke multiple microservices and aggregate the data payload
- E.g. Netflix API Gateway gets requests from
 - Televisions
 - Smartphones
 - gaming systems
 - Tablets

Handling large number of services

- Can you remember the name or ID of 50 services?
- How would you figure out if a service is present or not?

Solution:

- Maintain a registry for all the services –Registry Service
- Help to discover a service



Is there any standard for such communication ?

Is there any standard for such communication ?

- OpenAPI

OpenAPI

- A standard way to describe REST APIs
 - A specification for describing, consuming and visualizing RESTful web services
- Originally known as the Swagger Specification
- OpenAPI Configuration Document
 - ***Or*** openAPI definition
 - Contains more than one definition files
 - Uses YAML or JSON
 - Can be validated, tested and ported

OpenAPI

- A standard way to describe REST APIs
 - A specification for describing, consuming and visualizing RESTful web services
- Originally known as the **Swagger** Specification
- OpenAPI Configuration Document
 - *Or* openAPI definition
 - Contains more than one definition files
 - Uses YAML or JSON
 - Can be validated, tested and ported

Swagger

- <https://swagger.io/>
- An open source and professional tool
- A tool that
 - Simplify API development
 - for users, teams, and enterprises
 - help you design and document your APIs

Swagger/OpenAPIConfiguration document

Configuration Document:

- API Endpoint
 - E.g. */user,/upload,/devices, /data, /openstack-instances, /openstacktenant-flavors, /openstacktenant-images, /openstacktenant-instances*, etc.
- Operation for each endpoint
 - E.g. ***GET,POST,PUT,DELETE***
- Input Parameter
 - E.g. ***device_id=sens-897***
- Responses and format

Swagger Configuration -Example

```
1 openapi: 3.0.1
2 info:
3   title: Swagger SmartBuilding App
4   description: Smart building application
5   termsOfService: http://swagger.io/terms/
6   version: 1.0.0
7 servers:
8   - url: https://Devicestore.swagger.io/v2
9   - url: http://Devicestore.swagger.io/v2
10 paths:
11   /Device:
12     post:
13       tags:
14         - Device
15       summary: Add a new Device to the store
16       operationId: addDevice
17       requestBody:
18         description: Device object that needs to be added to the store
19         content:
20           application/json:
21             schema:
22               $ref: '#/components/schemas/Device'
23             required: true
24       responses:
25         405:
26           description: Invalid input
27           content: {}
28       security:
29         - Devicestore_auth:
30           - write:Devices
31           - read:Devices
32       x-codegen-request-body-name: body
```

API definition *must include* the *version* of the OpenAPI Specification that this definition is based on

The ***servers*** section specifies the API server and base URL.

Swagger Configuration -Example

```
1 openapi: 3.0.1
2 info:
3   title: Swagger SmartBuilding App
4   description: Smart building application
5   termsOfService: http://swagger.io/terms/
6   version: 1.0.0
7 servers:
8   - url: https://Devicestore.swagger.io/v2
9   - url: http://Devicestore.swagger.io/v2
10 paths:
11   /Device:
12     post:
13       tags:
14         - Device
15       summary: Add a new Device to the store
16       operationId: addDevice
17       requestBody:
18         description: Device object that needs to be added to the store
19         content:
20           application/json:
21             schema:
22               $ref: '#/components/schemas/Device'
23             required: true
24       responses:
25         405:
26           description: Invalid input
27           content: {}
28       security:
29         - Devicestore_auth:
30           - write:Devices
31           - read:Devices
32       x-codegen-request-body-name: body
```

- **PATH:** The paths section defines individual endpoints (paths) in your API, and the HTTP methods
- Here it will be `<url>/Device`
- A single path can support multiple operations

Example:

https://api.example.com/v1/users?role=admin&status=active		
\	/\	/ \
server URL	endpoint	query parameters
path		

Swagger Configuration -Example

```
1  openapi: 3.0.1
2  info:
3    title: Swagger SmartBuilding App
4    description: Smart building application
5    termsOfService: http://swagger.io/terms/
6    version: 1.0.0
7  servers:
8    - url: https://Devicestore.swagger.io/v2
9    - url: http://Devicestore.swagger.io/v2
10 paths:
11   /Device:
12    post: .....
13     tags:
14     - Device
15     summary: Add a new Device to the store
16     operationId: addDevice
17     requestBody:
18       description: Device object that needs to be added to the store
19       content:
20         application/json:
21           schema:
22             $ref: '#/components/schemas/Device'
23           required: true
24     responses:
25       405:
26         description: Invalid input
27         content: {}
28     security:
29       - Devicestore_auth:
30         - write:Devices
31         - read:Devices
32     x-codegen-request-body-name: body
```

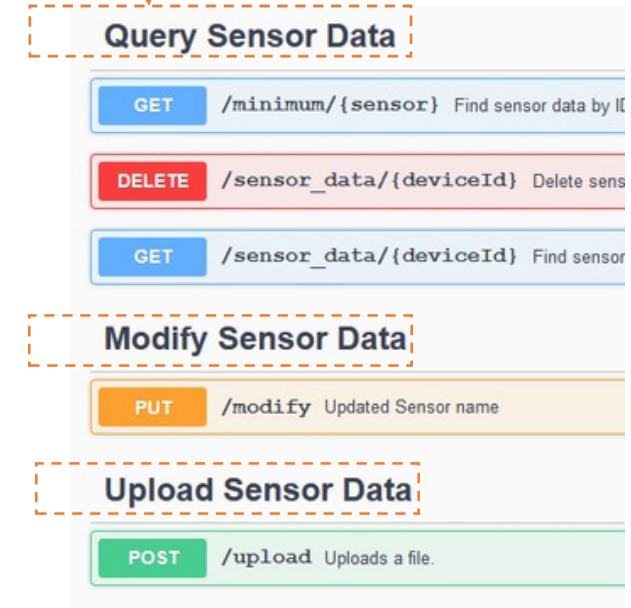
POST: An operation definition includes parameters, request body (if any), possible response status codes (such as 200 OK or 404 Not Found) and response contents.

Swagger Configuration -Example

```
1  openapi: 3.0.1
2  info:
3    title: Swagger SmartBuilding App
4    description: Smart building application
5    termsOfService: http://swagger.io/terms/
6    version: 1.0.0
7  servers:
8    - url: https://Devicestore.swagger.io/v2
9    - url: http://Devicestore.swagger.io/v2
10 paths:
11   /Device:
12     post:
13       tags: .....
14       - Device
15       summary: Add a new Device to the store
16       operationId: addDevice
17       requestBody:
18         description: Device object that needs to be added to the store
19         content:
20           application/json:
21             schema:
22               $ref: '#/components/schemas/Device'
23             required: true
24       responses:
25         405:
26           description: Invalid input
27           content: {}
28       security:
29         - Devicestore_auth:
30           - write:Devices
31           - read:Devices
32       x-codegen-request-body-name: body
```

TAG– used to group operations logically by resources or any other qualifier

These are coming from Tags



Swagger Configuration -Example

```
1  openapi: 3.0.1
2  info:
3    title: Swagger SmartBuilding App
4    description: Smart building application
5    termsOfService: http://swagger.io/terms/
6    version: 1.0.0
7  servers:
8    - url: https://Devicestore.swagger.io/v2
9    - url: http://Devicestore.swagger.io/v2
10 paths:
11   /Device:
12     post:
13       tags:
14         - Device
15       summary: Add a new Device to the store
16       operationId: addDevice
17       requestBody: ◀ .....
18         description: Device object that needs to be added to the store
19         content:
20           application/json:
21             schema: ◀ .....
22               $ref: '#/components/schemas/Device'
23             required: true
24       responses:
25         405:
26           description: Invalid input
27           content: {}
28       security:
29         - Devicestore_auth:
30           - write:Devices
31           - read:Devices
32       x-codegen-request-body-name: body
```

Request bodies are typically used with “create” and “update” operations (POST, PUT, PATCH). For example, when creating a resource using POST or PUT, the request body usually contains the representation of the resource to be created.

The Schema Object allows the definition of input and output data types.

Swagger Configuration -Example

```
1  openapi: 3.0.1
2  info:
3    title: Swagger SmartBuilding App
4    description: Smart building application
5    termsOfService: http://swagger.io/terms/
6    version: 1.0.0
7  servers:
8    - url: https://Devicestore.swagger.io/v2
9    - url: http://Devicestore.swagger.io/v2
10 paths:
11   /Device:
12     post:
13       tags:
14         - Device
15       summary: Add a new Device to the store
16       operationId: addDevice ◀.....
17       requestBody:
18         description: Device object that needs to be added to the store
19         content:
20           application/json:
21             schema:
22               $ref: '#/components/schemas/Device'
23             required: true
24       responses:
25         405:
26           description: Invalid input
27           content: {}
28       security:
29         - Devicestore_auth:
30           - write:Devices
31           - read:Devices
32       x-codegen-request-body-name: body
```

operationId is an optional unique string used to identify an operation. Use this value to name the corresponding methods in code

The Schema Object allows the definition of input and output data types.

Swagger Configuration -Example

```
1  openapi: 3.0.1
2  info:
3    title: Swagger SmartBuilding App
4    description: Smart building application
5    termsOfService: http://swagger.io/terms/
6    version: 1.0.0
7  servers:
8    - url: https://Devicestore.swagger.io/v2
9    - url: http://Devicestore.swagger.io/v2
10 paths:
11   /Device:
12     post:
13       tags:
14         - Device
15       summary: Add a new Device to the store
16       operationId: addDevice
17       requestBody:
18         description: Device object that needs to be added to the store
19         content:
20           application/json:
21             schema:
22               $ref: '#/components/schemas/Device'
23             required: true
24       responses:
25         405: ◀.....
26           description: Invalid input
27           content: {}
28       security:
29         - Devicestore_auth:
30           - write:Devices
31           - read:Devices
32       x-codegen-request-body-name: body
```

- Each operation must have at least one response defined, usually a successful response.
- defined by its HTTP status code and the data returned in the response body and/or headers

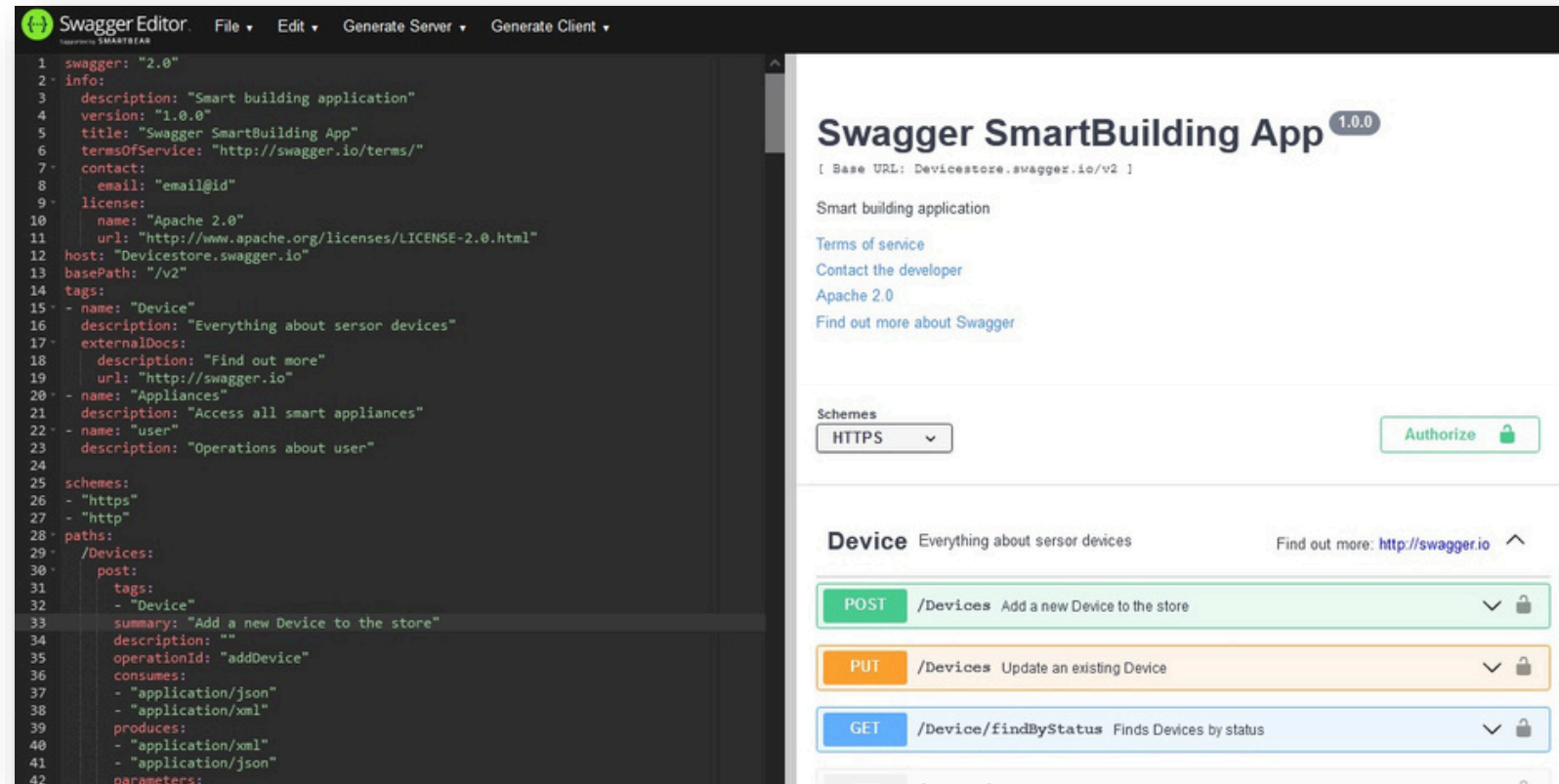
Swagger Editor

Open source tools

- ***Swagger Editor***
- ***Swagger UI***

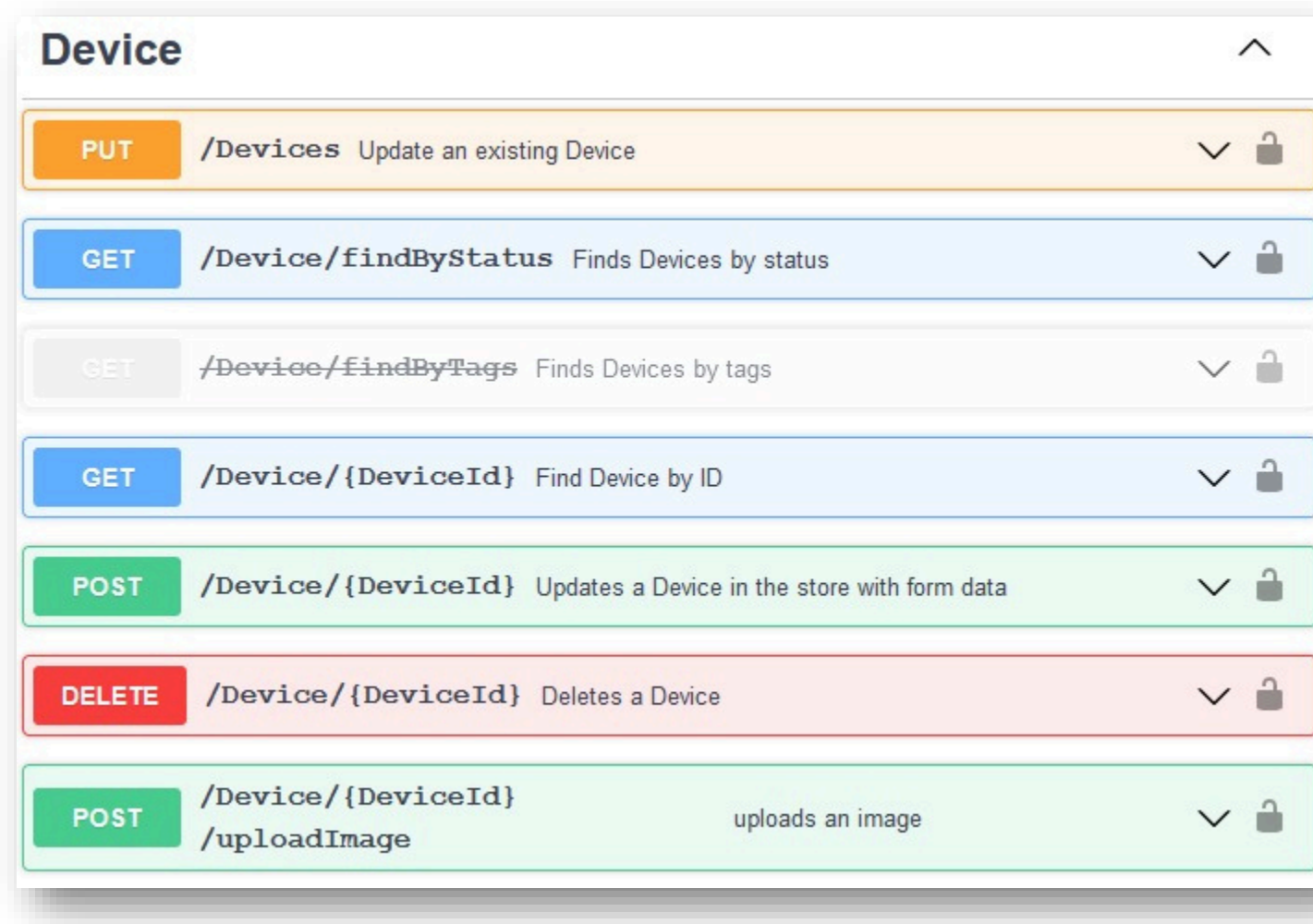
Swagger Editor

- Open source tools
- **Swagger Editor** –
 - Design, describe, and document your API
- Validate your syntax
- Instant Visualization



Swagger UI

- Open source tools
- *Swagger Editor*
- **Swagger UI** –
 - to visualize and interact with the API's resources without having any of the implementation logic in place.
 - Documentation Site generator



So far...

- How microservices communicate?
- Communication protocols
- OpenAPI standard
- Swagger tool

So far...

- How microservices communicate?
- Communication protocols
- OpenAPI standard
- Swagger tool

Do you see any relationship between
microservice and ***containers***?

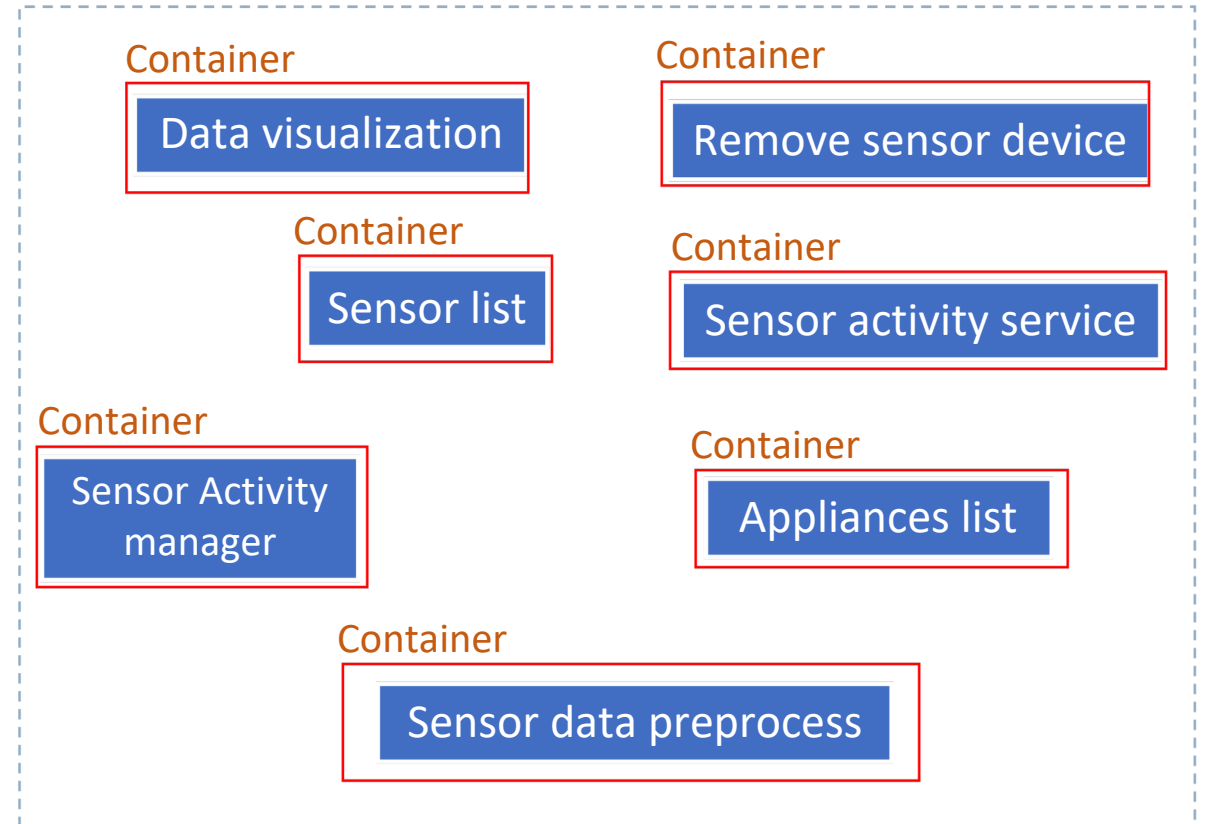
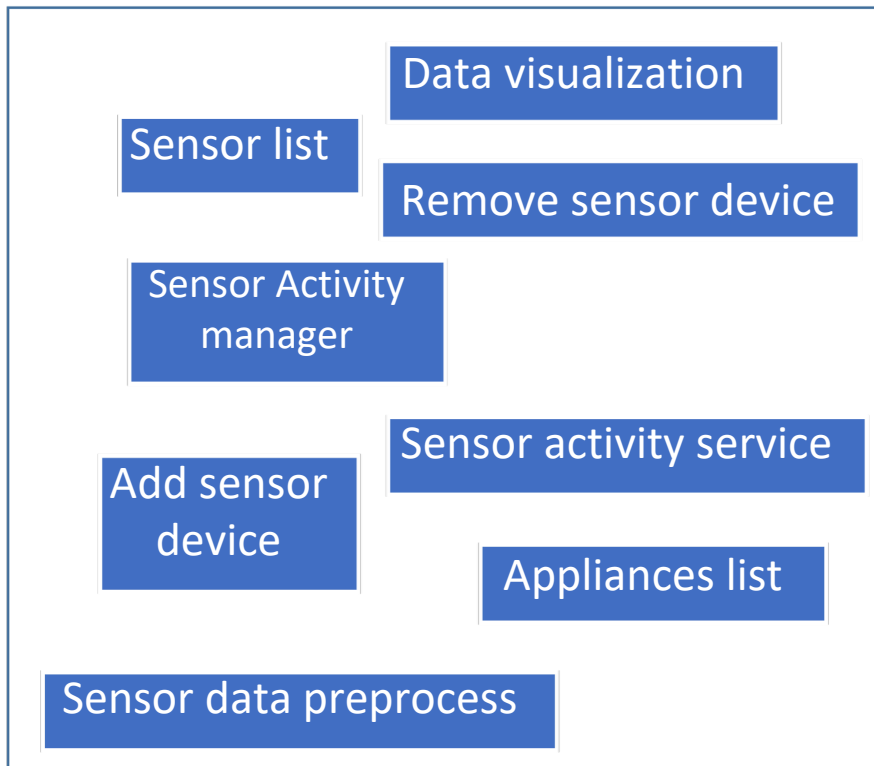
Container and Microservice

- ***Microservices*** are about the design of software.
- ***Containers*** are about packaging software for deployment.

Container and Microservice

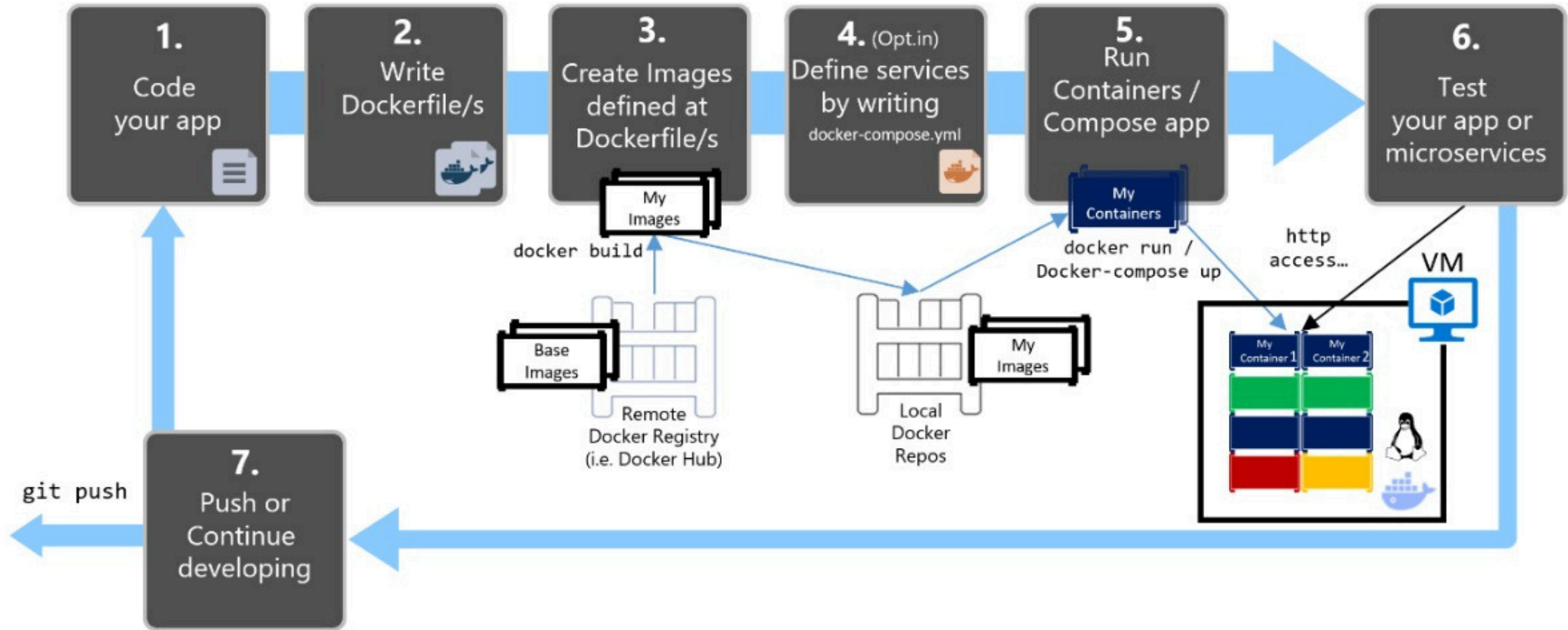
- You can use a container for hosting a microservice.
- Run microservices within containers.

VM 1



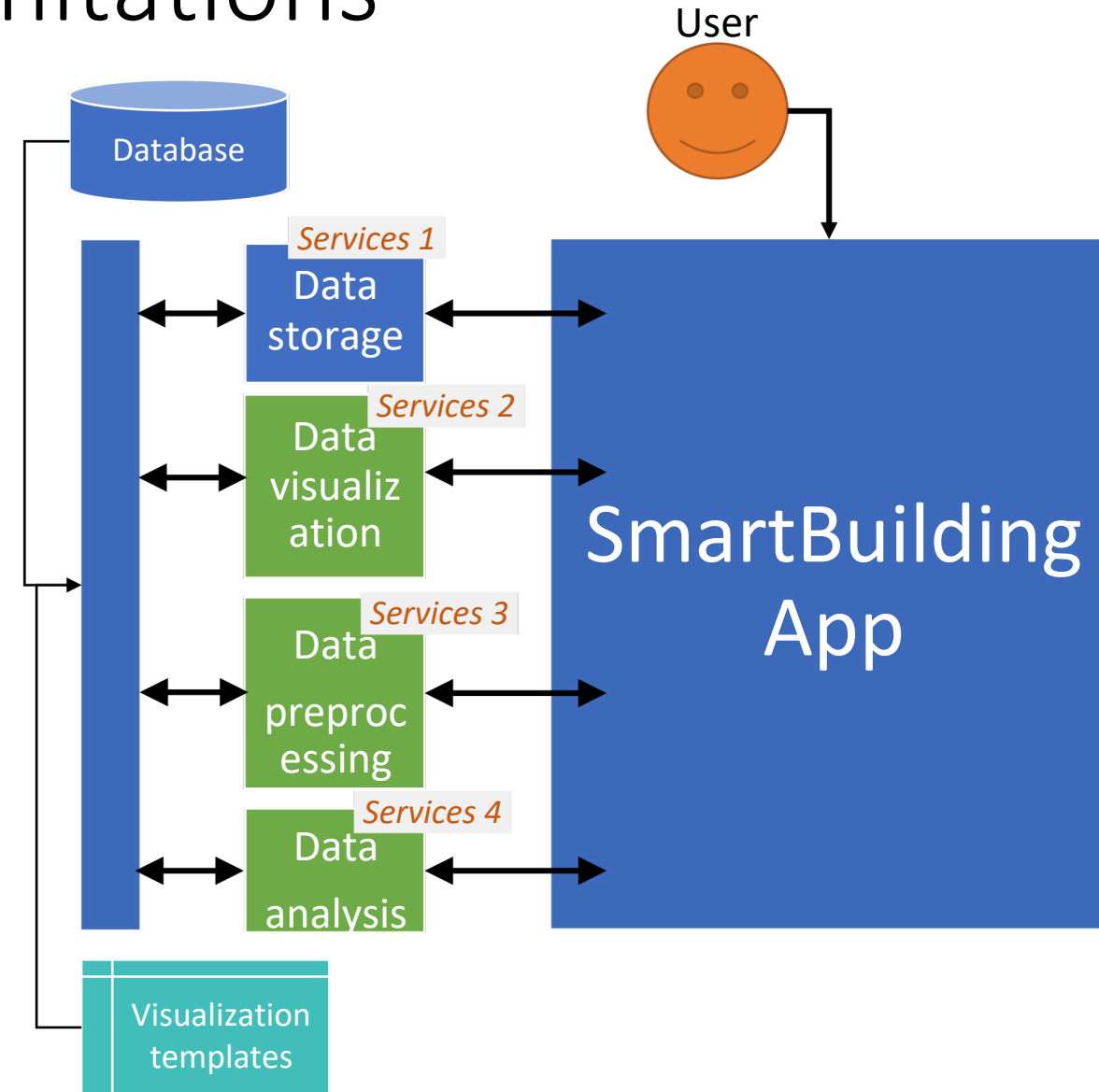
Container and Microservice

Inner-Loop development workflow for Docker apps



Microservice Approach –Limitations

- Development Complexity (whole application)
- Difficult to deploy
 - Need to manage the deployment of each service (instead of only the bigger application)
- Distributed data management
 - Each service has its own database (private)
 - Maintaining data consistency is a challenge
 - Querying is a challenge
- Debugging hundreds of services
 - tracing the bugs and errors
- Testing:
 - Need to test each service:
 - Dependencies
 - Communications
 - Input/output parameters



Microservice Approach –Limitations

